

Estructura de memoria

Programación I

UNRN

Universidad Nacional
de Río Negro

17

08

INGCOM

2026

Repaso de la clase anterior:

- Definición de función (ejemplos)
- Formas básicas para llamar a una función
- Variables globales vs variable locales
- variable tipo Static
- Pasaje de argumentos
- Documentación: TIPS
- Prototipo de una Función
- Pre_condicion vs post_condicion

Hagamos un ejercicio de ejemplo

Se nos pide implementar el software de una alcancía electrónica sencilla. El sistema debe poder:

- Depositar dinero.
- Retirar dinero.
- Consultar el saldo.

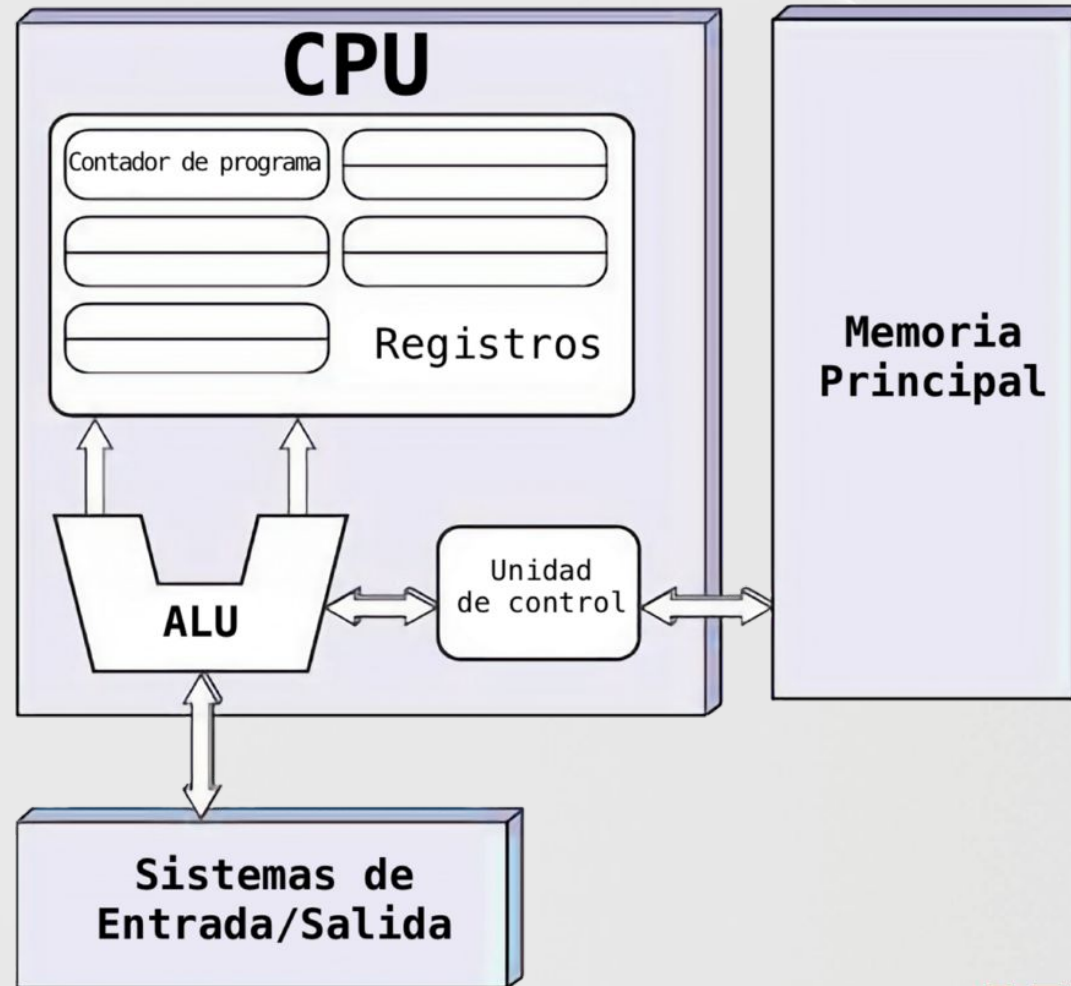
Implementarlo como un programa que tiene un menú de operaciones y termina cuando el usuario presiona la 'S' o la 's'.

Debe controlar que no se retire más dinero del que hay ni que se depositen cantidades negativas de dinero.

Hagamos un ejercicio de ejemplo

1. Plantear un diseño del sistema en C.
2. Implementar funciones para consulta de saldo y depósito.
3. Documentar y establecer pre y post condiciones.

Arquitectura de Von Neumann



Gestión de Memoria en C

- **La memoria no es un bloque plano:**

- Para ejecutar un programa, el SO organiza la RAM en segmentos especializados.
- Cada segmento tiene propósitos, permisos y ciclos de vida distintos.

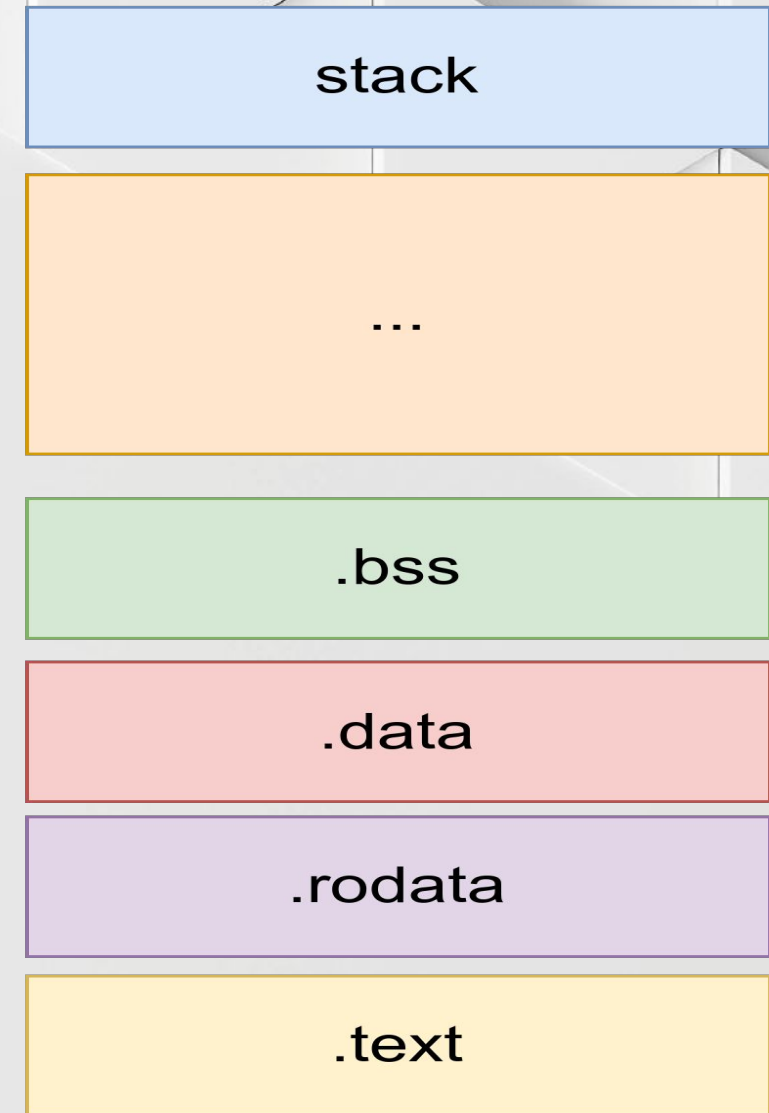
- **Objetivos de la clase:**

- Comprender dónde reside cada variable.
- Visualizar el crecimiento de la pila (Stack).
- Identificar errores comunes de acceso.

Estructura General del Programa

- **Jerarquía de Segmentos:**

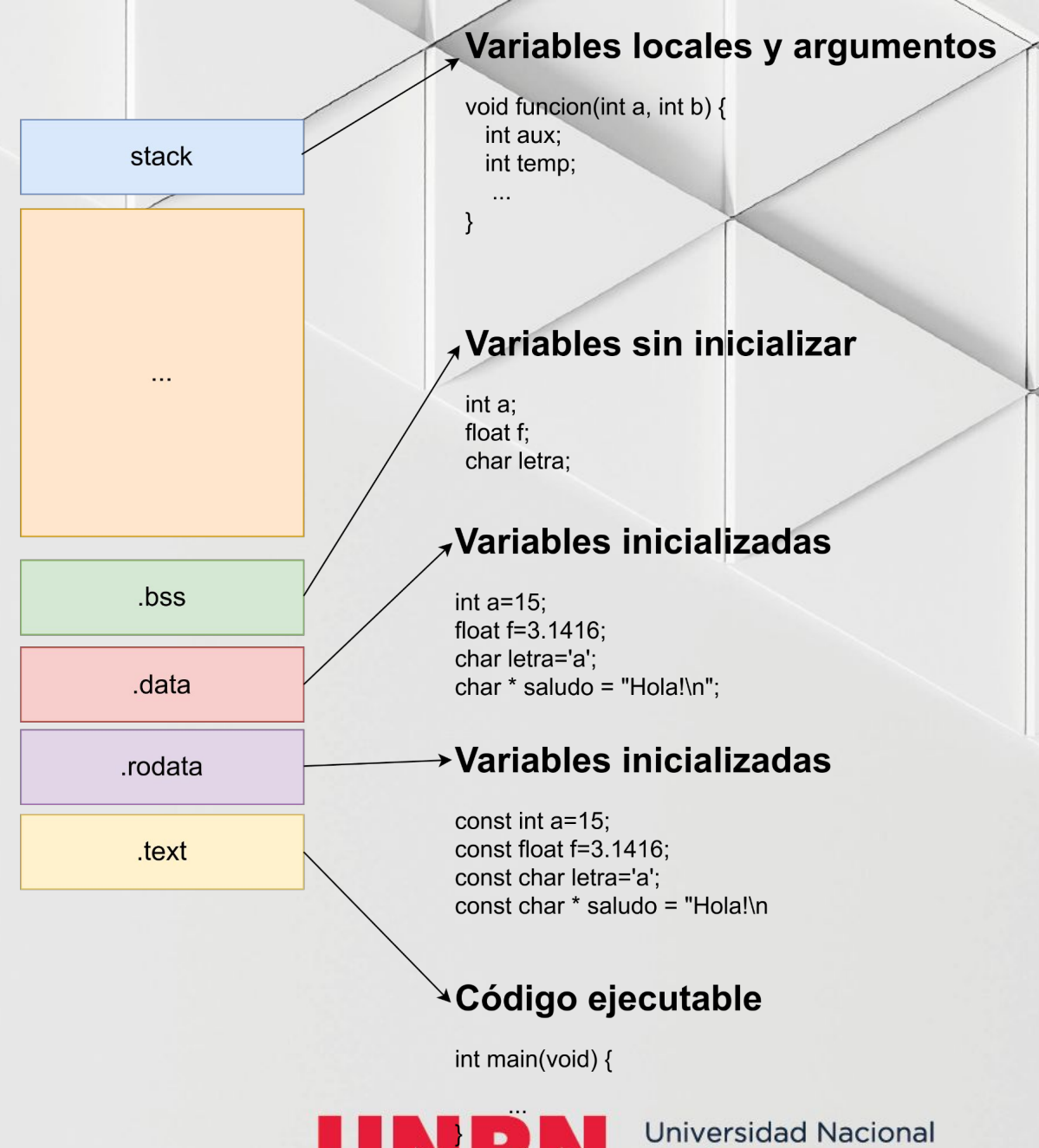
- **Stack:** Memoria automática (crece hacia abajo).
- **Heap:** Memoria dinámica (crece hacia arriba).
- **Data/BSS:** Variables globales y estáticas.
- **Text:** El código binario (Solo lectura).



Estructura General del Programa

- **Jerarquía de Segmentos:**

- **Stack:** Memoria automática (crece hacia abajo).
- **Heap:** Memoria dinámica (crece hacia arriba).
- **Data/BSS:** Variables globales y estáticas.
- **Text:** El código binario (Solo lectura).



Segmento de Texto (Code)

- **Contenido:**

- Almacena las instrucciones ejecutables del programa (binario).

- **Características:**

- Es de **solo lectura** para evitar que el programa se modifique a sí mismo.
- Es de tamaño fijo una vez que el programa se carga.

- **Ubicación:**

- Se sitúa en la parte más baja de la memoria para protegerlo de desbordamientos del Stack/Heap.

Segmento de Datos y BSS

- **Data Segment:**

- Variables globales e inicializadas por el programador.

- **BSS (Block Started by Symbol):**

- Variables globales declaradas pero no inicializadas (el SO las pone en 0).

- **Ejemplo en C:**

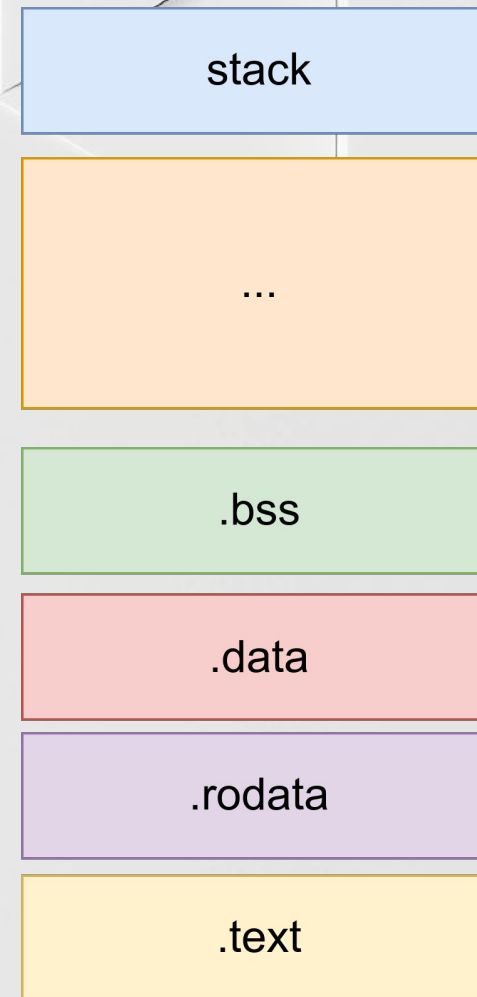
```
int global_ini = 10; // Segmento DATA
int global_no_ini; // Segmento BSS
int main()
{
    ...
    return 0;
}
```

Ubicación de Variables (Casos)

```
#include <stdio.h>
int g = 100; // ??
float f;
```

```
void func()
{
    int local = 5; //???
}

int main()
{
    static int s = 20; // ????
    int x = 10; // ????
    return 0;
}
```

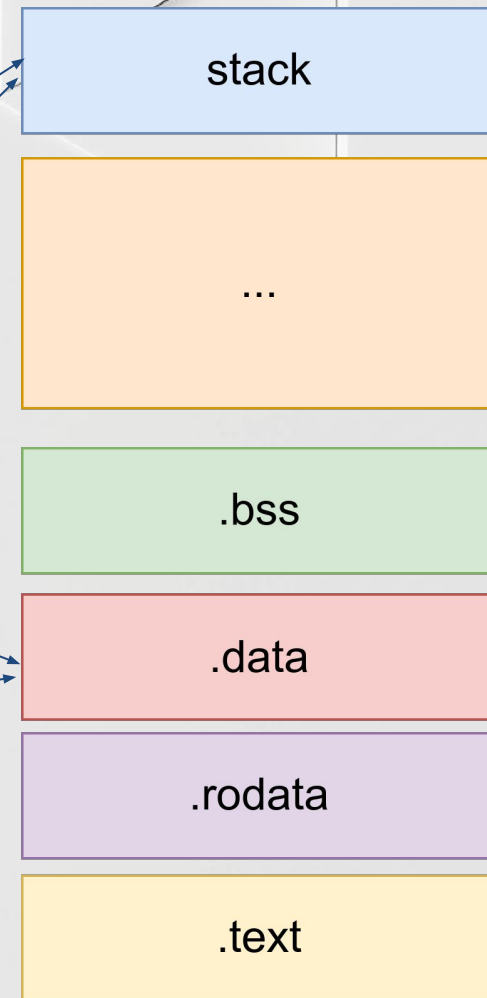


Ubicación de Variables (Casos)

```
#include <stdio.h>
int g = 100; // ??
float f;
```

```
void func()
{
    int local = 5; //???
```

```
int main()
{
    static int s = 20; // ???
    int x = 10; // ???
    return 0;
}
```



¿Dónde van las Constantes?

- **Constantes Literales de Cadena:**

- `char *p = "Hola";` -> La cadena "Hola" va en el segmento de **Texto/Read-Only Data**.

- **Variables `const`:**

- Si son locales: van al Stack, pero el compilador prohíbe su edición.
- Si son globales: Suelen ir a un segmento de solo lectura dentro de DATA.

- **Ejemplo:**

```
const int MAX = 100; // Segmento Data (Read-only)
int main()
{
    const int local_c = 50; // Stack
}
```

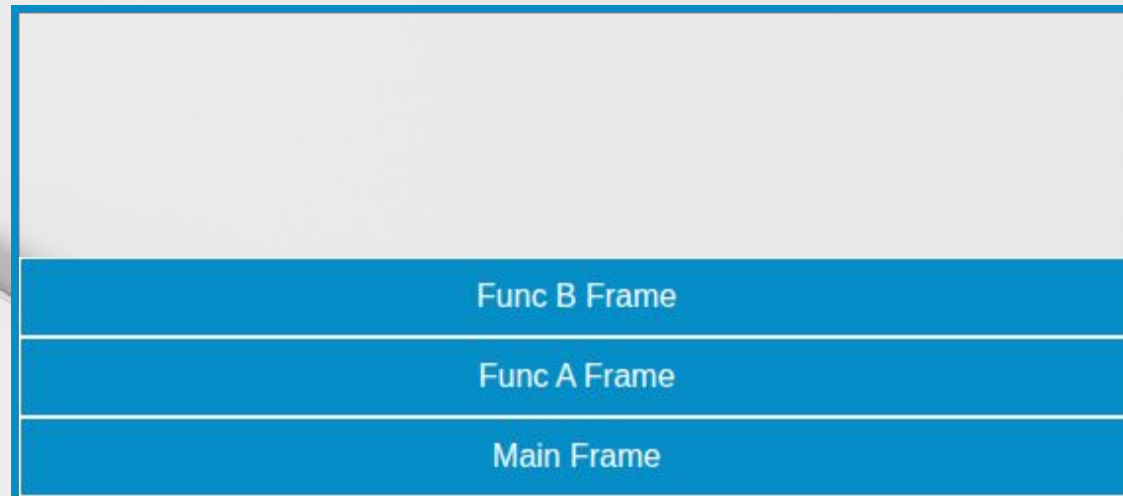
La Pila de Llamadas (Stack)

- **Mecánica LIFO:**

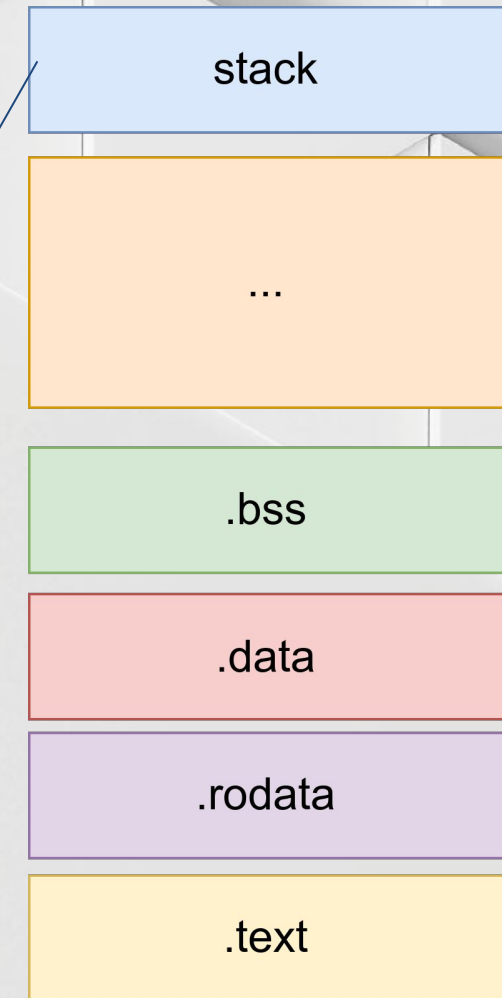
- Last-In, First-Out (Último en entrar, primero en salir).

- **Crecimiento:**

- Se expande hacia direcciones de memoria menores al llamar funciones.
- Se contrae automáticamente al retornar (return).



Estado actual: Main calls A, A calls B



Stack Frames y Parámetros

- **¿Qué hay en un Stack Frame?**

- Argumentos/Parámetros de la función.
- Dirección de retorno (¿A dónde vuelvo tras el return?).
- Variables locales.

- **Ejemplo de Proceso:**

```
void suma(int a, int b)
{
    // 'a' y 'b' se crean en el stack
    int res = a + b;
    // 'res' se crea en el stack
}
```

Visualización del Crecimiento

```
void B()
{
    int z;
}
void A()
{
    int y;
    B();
}

int main()
{
    int x;
    A();
}
```



Cada bloque es un **frame**. Al terminar B, su espacio queda disponible inmediatamente.

Ciclo de Vida en Memoria

- **Paso a paso:**

- 1. Se llama a la función -> El SP (Stack Pointer) se desplaza.
- 2. Se reserva espacio para las locales.
- 3. Se ejecutan instrucciones.
- 4. Al cerrar la llave }, el SP vuelve a su posición anterior.

- **Importante:**

- Los datos no se "borran" físicamente (0), simplemente se consideran inválidos/libres.

Código que rompe (SegFault)

Aclaración: El SO detecta el acceso ilegal y mata el proceso por seguridad.

Faltan algunas variables:

- *Vectores estáticos.*
- *Vectores VLA.*
- *Memoria dinámica.*

La próxima....

FIN